

Ch. 10

迴圈結構與重複執行

Horazon

應用程式設計

本章目標

1. 理解 **迴圈 (Loop)** 的概念。
2. 學習 **計次迴圈 (For Loop)**。
3. 學習 **條件迴圈 (While Loop)**。
4. 實作：累加計算機 ($1+2+\dots+N$)。
5. 挑戰：九九乘法表 (巢狀迴圈)。

為什麼需要迴圈？

試想一個問題：

「請寫一個程式，算出 1 加到 10 的總和。」

```
sum = 1 + 2 + 3 + 4 + 5 + 6 + 7 + 8 + 9 + 10
```

這還算簡單，手拉積木只要拉 10 次。

如果是 1 加到 1000 呢？

難道你要拉 1000 個積木嗎？

當然不可能！

這時候我們需要「迴圈 (Loop)」。

告訴電腦：

「這件事情幫我重複做 1000 次，每次數字加 1。」

這就是程式設計最強大的地方：**自動化重複工作**。

1. 計次迴圈 (For Each Number)

這是最常用的迴圈，在 **Control (黃色)** 抽屜裡：

```
for each number from [ 1 ] to [ 5 ] by [ 1 ]  
do  
    (這邊放要重複做的事)
```

參數解讀

- **number (變數)**：這是這一圈的數字 (例如第 1 圈是 1)。
- **from (起始值)**：從多少開始數。
- **to (終止值)**：數到多少結束 (包含這個數字)。
- **by (增量)**：每次加多少 (通常是 1)。

實作：累加器 (Sum 1 to N)

畫面設計

- 輸入框: `txt_n` (輸入 N)
- 按鈕: `btn_calc` (計算)
- 結果標籤: `lbl_result`

變數準備

我們需要一個變數 `sum` 來存總和。

- 一開始一定要設為 0！(這叫變數初始化/歸零)

程式邏輯 (Accumulator)

```
當 btn_calc.Click  
    設 sum 為 0 (重要! 如果不歸零, 下次按會繼續加上去)  
  
    for each i from 1 to (txt_n.Text) by 1  
        設 sum 為 ( get sum + get i )  
  
    設 lbl_result.Text 為 get sum
```

追蹤看看 (Trace Code)

假設 $N = 3$:

1. $i=1$: $\text{sum} = 0 + 1 = 1$
2. $i=2$: $\text{sum} = 1 + 2 = 3$
3. $i=3$: $\text{sum} = 3 + 3 = 6$
4. 結束 -> 答案 6。

2. 條件迴圈 (While Loop)

有時候我們不知道要跑幾次，只知道「什麼時候停」。

例如：

- **擲骰子**，直到擲出 6 點為止。(可能 1 次，也可能 100 次)
- **輸入密碼**，直到正確為止。

這時候要用 `while test do` 積木。

```
while (骰子點數 ≠ 6)  
do 重新擲骰子
```

小心無窮迴圈 (Infinite Loop)：如果條件永遠成立，程式會卡死！

3. 巢狀迴圈 (Nested Loop)

迴圈裡面可以再包一個迴圈嗎？**可以！**

經典範例：九九乘法表

```
For i from 1 to 9
  For j from 1 to 9
    Print (i * j)
```

- 當 `i=1` 時，`j` 會跑 1~9 (印出 1x1 ... 1x9)
- 當 `i=2` 時，`j` 又跑 1~9 (印出 2x1 ... 2x9)
- ...
- 總共會跑 $9 \times 9 = 81$ 次！

實作挑戰：倒數計時器

For 迴圈不一定要從小數到大，也可以從大數到小！

設定

- from: 10
- to: 1
- by: -1 (負數代表倒扣)

應用

- 火箭發射倒數 10, 9, 8...
- 炸彈倒數計時。

進階觀念：Break (中斷)

有時候迴圈跑到一半，我們想提早離開。

例如：從 1 數到 100，但如果數字是 50 就停下來。

AI2 提供 `break` 積木 (在 Control 裡)。

```
for each number from 1 to 100
  if number = 50
    break (跳出迴圈)
```

針對清單的迴圈 (For Each Item)

除了數數字，迴圈最常用的是「把清單裡的東西全部跑一遍」。

```
for each item in list (你的清單)  
do  
    (對每一個 item 做事)
```

我們下一章講清單的時候，會大量用到這個！

重點回顧

1. For Loop: 固定次數的重複 (1 to N)。
2. While Loop: 條件式的重複 (直到...為止)。
3. Accumulator: `sum = sum + i` 是累加神器。
4. Initialization: 進入迴圈前，變數記得歸零。
5. Nested Loop: 迴圈包迴圈，處理二維資料 (如乘法表、座標)。